Solace: A Technical White Paper

Oct 6, 2026 · Aaron

Summary

Solace is an agentic development environment. It wraps a software project and mediates all work on it: AI agents carry out the engineering tasks, and a human sets direction, answers the agents' questions and approves the results. Agents change the project only through Solace, and the human assigns, follows and reviews their work through the app, without reading code or using a command line.

Solace keeps the project's code in a store of symbols (functions, types and constants), each with a permanent ID. An agent reads and changes code by symbol, through a single tool, and every change is type-checked as it arrives; code with errors is not saved. Solace writes ordinary source files from the store for the normal compiler and test runner. The agent has no shell and no file tools. It reaches the project's other files through the same tool, under rules set by file type, and runs only the commands the human has declared. Each task records what it changed, so its work can be reviewed and undone.

The human works in three places: a task board, where tasks are assigned to agents, questions are answered, runs are followed live and results are reviewed; a contract view, which draws the project's entry points, logic, side effects, tests and files and describes each function in plain words; and a migration tool, which brings an existing project under Solace and can be reversed.

The first goals are the guardrails on agents and the interface for humans. How many tokens an agent uses is measured and reduced, but never at the expense of those two.

Solace is at an early stage. It ships as a single executable that runs locally for one person, and it supports projects written in Gleam, a functional language that runs on Erlang's virtual machine (the BEAM) and on JavaScript. Running functions from the contract view, replaying runs to compare a change's behavior, a strategy page for planning, and possibly a language of Solace's own are planned.

1. The problem

Development tools are built around people editing text files and reading what the compiler prints. That works poorly for an agent, and it leaves the person who directs the agent with little to go on besides the agent's own report.

For the agent

- **Finding its way around.** It searches, opens and reads files for several rounds before changing anything, and it re-sends its whole conversation to the model on every step.
- **Editing text.** Find-and-replace patches fail when whitespace differs or the search text appears more than once.
- **Remembering.** Each session starts with no record of earlier plans or decisions.

- **Hidden inputs.** Time, randomness and network calls don't appear in function signatures, so behavior is hard to predict and tests can be flaky.

For the human

- **Review means reading code.** A person who owns a project but doesn't read its language, or who has no time to read every change, can only trust the agent's summary of what it did.
- **Work is hard to follow.** Running several agents on a project means watching terminals, and the plan, the decisions and the questions agents have are scattered.
- **Agents can reach everything.** An agent with a shell and file tools can change files it was never asked to touch, including the tests that would catch its mistakes.
- **Planning happens elsewhere.** The vision, the feature list and the roadmap live in documents and trackers that agents don't see.

2. What Solace is

Solace wraps a user's project. Agents change the project only through Solace, and humans direct and review the work through Solace's app. All of it ships as one executable, `solace`, which holds the store, the command-line interface, the server for agents, the task board with its web client, and the generator for locked-down agent configurations. It runs on the user's machine and keeps its data in the project's `.solace/` folder. There are no accounts.

Solace contains no model. It starts agent programs that the user has already installed and signed in to, currently the Codex CLI for OpenAI models and Claude Code for Claude models, and stores no credentials. Any OpenAI-compatible or Claude-compatible endpoint, including a model running on the same machine, can be added in a settings file.

Tool	For	Status
Agent tool	Agents read and change the project by symbol, with type checking, impact analysis and test selection	**Built**
Task board	Humans create tasks, assign them to agents, answer questions, follow runs live and review the result	**Built**
Contract view	Humans see what the software does, what it touches and what tests cover it, and can run it	**Built**
Migration tool	Brings an existing project under Solace, with checks and a way back	**Built**
Multi-project roots	Several projects on one board, with tasks that change more than one	**Built**
Strategy page	Vision, feature design, roadmap with milestones, breakdown into tasks	**Planned**
Further life-cycle tools	The rest of the software engineering life cycle	**Planned**

3. How humans work

The human side is the Solace app. It is meant for someone who gives creative input and judgment and does not need to read source code or use a terminal.

3.1 The task board

Tasks move through columns, and the workflow is configurable. A person writes a task in a markdown editor, which can include images that agents also see. In planning, the person can have an agent investigate the task first. An investigation changes nothing and reports findings or a plan. Then the person assigns the task to a model and an effort level. Solace starts the agent and puts the agent's notes and questions on the task. It moves the task to *blocked* when the agent needs an answer and to *needs review* when it has finished. Cards show which tasks have an agent working, and each run can be followed live: the agent's messages and each command it ran with its result.

In review, a separate agent checks the work, and the person approves it or returns the task for fixes with new instructions. Solace records what each task changed and the state before it, so a person can undo a task entirely, unless a later task has changed the same code. A built-in guide explains the workflow.

3.2 The contract view

The contract view shows a project as a drawn graph of its entry points, its logic, the effects it has on the outside world (files, network, clock, randomness, other processes), its tests and its other files. Each node and connection is labeled with how it is known: written in the code, derived from reading it, or observed in a run. The view never presents a derived fact as a guarantee. A person can open any part, read it in plain words (what it takes, what it gives and what it touches), and run any public function with a form built from its types.

For a task, the view shows what changed in terms of behavior: the tests that ran, what the change reaches that no test covers, and a before-and-after replay of the same run. A difference is shown as evidence for the person to judge, not as a failure.

3.3 Migration

To bring an existing Gleam project under Solace, the person chooses **Migrate this project** on the board. Solace copies the folder, builds and tests the original as a baseline, sets up the copy, regenerates the code from the store, and writes a report. The report covers whether every symbol was found, whether the regenerated code matches the original and where it differs, whether the tests still behave the same, whether any path points outside the folder, whether every file was kept, and whether the copy builds without Solace's files. The original folder is not changed until the person chooses to switch, which keeps the original as a backup folder and can be reversed. Where bubblewrap is installed, every command Solace runs during a migration sees the original as read-only.

3.4 Several projects on one board

One Solace root can hold several projects below it, shown as lanes on one board. The root folder is itself a project of plain files (documents, scripts and code in other languages), so agents can maintain

those too. A task can change several projects, chosen when it is created. Agents in different projects work at the same time.

3.5 Planned: strategy and planning

Planned, not built.

The board starts at a task. The strategy page will start earlier, with the project's purpose, and carry it down to tasks:

- **Vision.** A person writes down what the software is for and who uses it.
- **Feature design.** A person and an agent work out a feature together: what it does, its edge cases and the questions that need an answer before work starts.
- **Roadmap and milestones.** Features are placed on a roadmap with milestones.
- **Breakdown.** An agent proposes tasks from a feature, the person edits them, and they go onto the board.

The aim is one place where a person plans with agents, watches work happen and reviews the product, instead of a planning tool, a tracker and a terminal.

4. How agents work

4.1 The data model

Everything in a project's code is a symbol: a function, type, type variant, constant or test. Each symbol has a permanent ID that is assigned by the system and never reused, a name, a revision, and a history of changes. Editing or renaming a symbol keeps its ID, and deleting is refused while anything still uses the symbol.

The revision is a hash of the symbol's code without its comments and doc text, so editing only a comment does not count as a change. Solace records the result of each test together with a hash of everything the test can reach, and runs a test again only when something it reaches has changed.

The code lives in one SQLite file, `.solace/store.db`. Dependencies between symbols are worked out from their text and drive the "who uses this" queries. After each change, Solace writes the Gleam files back out for the normal compiler and test runner, and the agent never sees them. Edits made outside Solace (by hand, by another tool, or by pulling from Git) are read back into the store. If an agent is working, the edit joins its task. Otherwise it waits on an *Unexpected Changes* task for the person to keep or discard. Code that was approved in Solace, such as a teammate's pulled work, is recognized from a committed list and read in without a prompt.

4.2 The agent tool

The agent gets one tool, `solace`, provided over MCP (Model Context Protocol, the standard way coding agents call external tools). The tool takes `command` (one or more command lines, run in order, so several steps can share one round trip), `text` (new code, a patch, a note or a question) and `old`

(text to replace, for small edits). The agent's first message is a short workflow with examples and one line per command, and `help` gives every option. A typical session:

1. `session start --task` matches the words of the task against names, documentation and code, adds callers, called functions and covering tests, and returns about 2,000 to 3,000 tokens: each symbol's signature and description, why it was chosen, the project's `AGENTS.md` instructions, the other projects on the board, and notes from earlier sessions.
2. `inspect a b c --context` returns each symbol's code, the signatures of what it uses, its callers (with the line that calls it), and the smallest test that exercises it. For a type, it lists every place that constructs it with every field, which are the places a new field breaks. A signature is printed once per reply, and a reply over a size limit says what it left out and the command that shows it.
3. `submit` followed by `test`, in the same call. `submit` takes a patch in the `apply_patch` form that agents already know, naming symbols or modules instead of files, so the agent sends only the lines that change. Solace type-checks it and saves it. Missing imports are added. The reply reports what changed in the set of broken symbols: what it fixed, one line for each symbol still broken, and new errors in full.
4. `test` runs only the tests that can see a change since they last passed, plus any that failed. It reports failures only, and says what else the change reaches that no test checks.
5. `task done` ends the run with a summary for human review, and `work add` leaves a note for the next session.

Command	What it does
`session start [--task]`	Starting point: relevant symbols, instructions, other projects, saved notes
`inspect`, `view`, `refs`	Read symbols with their dependencies, callers and tests; module outlines; what uses a symbol or reads a record field
`find`, `grep`	Search code; search other project files
`impact`	What a change reaches, nearest first, with the tests that cover each
`submit`, `delete`	Change symbols by patch, by exact replacement or by whole block; remove an unused symbol
`drafts`	Rejected submits are kept, and the agent patches and retries one instead of resending it
`test`, `check`	Run the affected tests (or all of them); type-check everything
`probe`	Run throwaway code, start from an existing test's setup, or trace the calls a function receives, with arguments and results
`files`, `read`, `write`, `remove`	Project files outside the store, under the rules in 4.3
`build`, `format`, `run`	Build and format the project, and run commands the human declared
`task note`, `ask`, `diff`, `done`	On the board: add a note, ask the human a question with 2 to 4 options and wait, see what the task has changed, and finish
`work add`, `list`, `done`	Notes that carry over between sessions

Reading a dependency, such as a library module, gives its public signatures and documentation from the package source without storing them. `--project <folder>` on a read command reads another project on the same board, and on any command changes another project that the task covers.

If submitted code has errors, nothing is saved. If it is correct but breaks other code, such as callers of a changed signature, it is saved and the broken callers are listed until fixed. Errors carry a stable code, name the symbol, state what was expected and found, and group follow-on errors under the first. Agent programs differ in how much tool output they accept, so Solace sets the reply size limit and a few other settings for each agent it starts.

4.3 Guardrails

Solace sorts every file in the project into one of five categories, and the category decides what agents may do.

Category	Files	Agents
Graph	Gleam source and tests, held as symbols	Read and change symbols, never files
Watched	Other text files up to a size limit	Read and write through Solace, checked by any validator for that kind of file; undone with the task
Protected	`solace.toml`, `AGENTS.md`	Read; propose a change that the human approves or rejects
Unwatched	Build output, ignored files, binary or oversized files	Read and write through Solace
Internal	`.solace/` and any other Solace project nested inside	No access

Agents get no shell and no file tools, only the Solace tool and web access. They run only commands the human has declared in `solace.toml`, such as an end-to-end test suite. Reviews and investigations are read-only: they can't change code or files, and can run only declared commands that are marked as safe for reviews. An agent that needs something outside its task, for example a change to another project, asks the human with `task ask` instead of working around the limit.

5. Advantages

For agents

- **A short path to the right code.** The task-based starting list and the inspect bundle replace most of the search-open-read cycle.
- **Smaller reads.** The agent reads the definitions it needs, not whole files.
- **Reliable edits.** The agent changes a named symbol, so an edit can't fail on whitespace or land in the wrong place.
- **Fast feedback.** Each change is type-checked in about half a second, and reads take milliseconds.
- **Step-by-step refactors.** Broken callers are tracked, so a signature change can be fixed one caller at a time or submitted together with its callers.
- **Less bookkeeping.** Imports, formatting and rename fix-ups are never the agent's job.
- **Memory between sessions.** Saved notes appear at the next session start with the code they mention.

For humans

- **Review without reading code.** The contract view shows behavior, effects and test results, taken from the store and from recorded runs and not from the agent's account of itself.

- **A narrow, auditable interface.** With one tool, no file system and no shell, an agent can only work through Solace, every change is recorded, and a task can be undone as a whole.
- **Protected checks and settings.** Agents can't edit `solace.toml` or `AGENTS.md`, and changes they make outside Solace are surfaced for review.
- **One place for the work.** Tasks, agent questions, live runs and reviews are on one board, and agents can work on several projects at once.
- **No command line.** The human side runs in the app, and agents are started from the board.

6. Costs and open problems

- **One language.** Only Gleam projects are supported. The store and the contract view are written to be language-neutral, but nothing else has been tried.
- **Unfamiliar tools.** Models are trained on file-based workflows. They still sometimes open whole modules or read tests one at a time, which costs extra steps.
- **Reply size has to be tuned.** Agents re-send everything they have read on each step, so each reply from Solace adds to the cost of every later step. Solace keeps replies short: a symbol's signature is printed once, replies have a size limit, and the starting list names symbols and leaves out their signatures. The right amount of detail varies by task and model, so these limits are adjusted from benchmark results.
- **Friction with file-based tools.** Editors, linters and GitHub review depend on the Gleam files that Solace writes out. Edits made directly to those files have to be reviewed in Solace before agents continue.
- **Contract-view facts are derived.** Effects and dependencies are found by reading Gleam code. The view says so, but it can miss an effect that is hidden behind a call it doesn't recognize.
- **Early scale.** It has been tried on two benchmark projects and on Solace's own repository, with one user. Behavior on large codebases and with several people is untested.
- **Agents cost money.** Agent runs use the model provider's service. Solace reduces what each task uses and doesn't remove the cost.
- **Missing pieces.** The strategy page, conflict detection between several people's changes, and the rest of the life-cycle tools are not built.

7. Early results

The test project is an order-management service in Gleam (22 modules, about 30,000 tokens of code, 98 tests) with ten tasks: new features, bug fixes, a change touching many callers, a state-machine change and resuming half-finished work. Each task was done by the same agent in two ways, once with ordinary files and a shell and once through Solace only, and hidden tests graded the results. A step is one round trip to the model. Tokens are all tokens the model processed during the task, including the conversation re-sent on each step.

Latest runs: four tasks, three runs each

These used the Codex CLI with GPT-6-Sol at medium reasoning effort, on four of the ten tasks: a coupon feature, a partial-return rule, a tax-exemption change that touches many callers, and a business-day delivery estimate. Every run passed, 12 of 12 with files and 12 of 12 through Solace. The table gives the median run for each task. The two sides ran at different times, and Solace changed between them: the workflow now sends patches instead of whole symbols and prints less per reply.

Task	Tokens, files	Tokens, Solace	Change	Steps (files / Solace)	Seconds (files / Solace)
Coupon feature	228k	107k	−53%	11 / 7	85 / 59
Partial return	232k	95k	−59%	10 / 6	70 / 44
Tax exemption	330k	146k	−56%	12 / 7	102 / 87
Business-day estimate	153k	119k	−22%	8 / 7	58 / 49
Total of medians	**943k**	**468k**	**−50%**	**41 / 27**	**315 / 239**

- Through Solace the agent used half the tokens, a third fewer steps and a quarter less time. Output tokens fell 25%.
- The first edit came after a median of 4 to 5 steps both ways, so the saving comes after orientation, not before it.
- No run had a failed edit through Solace.

The same model on a local machine

To check that the gain doesn't depend on one provider, two runs of each of four tasks used a quantized open model served on the same machine and run through Claude Code. All 16 runs passed. Through Solace the agent used 727k tokens against 1.81M with files (−60%) across the median runs, and wall-clock time was 7% lower (727 seconds against 780).

Earlier full comparison: all ten tasks

The first complete comparison used GPT-6-Luna at maximum effort, one run per task per side. All 20 runs passed. Through Solace, the agent used 43% fewer tokens (3.12M against 5.51M) and 40% fewer steps (97 against 163). The saving per task ranged from 62% down to none, and no task cost more. Resuming work from a saved note used 52% fewer tokens. Wall-clock time was 35% longer through Solace in that run. Tool time was under 1% in both setups, so the difference was model time. Part of it was the model writing whole symbols where a file patch sends only the changed lines, and part may

have been provider speed, since the two sides ran an hour apart. Solace has since added patches by symbol, and the latest runs, which used a different model, were faster than with files.

Repeat runs of the same task vary by 15% or more, and the latest comparison has three runs per task and one model on one project, so these are early signs. The full ten-task comparison has not been repeated since the changes above. The benchmark harness and every run's logs are in the repository.

8. What's next

- **Now:** Continue improvements in UX of Board, Contract View, and Migration pages.
- **Next:** The strategy page: vision, feature design with an agent, roadmap and milestones, and a breakdown into tasks that land on the board. Alongside it, more of the review experience: guarantees and decisions described in business terms, scenario replay, and tracing a failure back to its cause.
- **Later:** Tools for the rest of the software life cycle, possible support for more languages than Gleam

9. A possible language of Solace's own

Solace's tools were built over Gleam so that they could be measured against ordinary files before any language work. A language designed for agents remains possible later. It would write into the program what an agent can't see in ordinary code, so the compiler can check it:

- **Declared effects.** A function could affect the outside world only through a value it was handed, such as a clock, a database or an HTTP client, and its signature would list the effects it uses. Tests would pass fakes, so they behave the same on every run.
- **Contracts and examples in the signature.** Preconditions, postconditions and examples would sit beside the function. Examples would run as tests.
- **Types that rule out wrong meanings.** No null and no exceptions, distinct types for different kinds of ID, restricted ranges of values, and units of measure.
- **Explicit unfinished code.** A typed hole would mark a gap with a question for a human. The rest of the program would still compile and run its tests.

The contract view would then show these as written guarantees instead of derived facts. Whether the language is worth building will be decided from how far the current tools get with Gleam.